instituto superior de engenharia de lisboa

# SoftKnow: Data Mining and Knowledge Discovery for MedKnow

Final Project – A and A1


Group D10
49751 Beatriz Filipe
53537 Ana Assis


Project developed as part of Machine Learning and Data Mining subject
MEIC and MEIM
Winter Semester 2025/2026

December of 2025

# Contents


**1 Introduction and Background** **2**

1.1 Introduction . . . 2

1.2 Objectives . . . 2

1.3 From Data to Knowledge . . . 2

**2 Database Design** **3**

2.1 Conceptual Model (ER Model) . . . 3

2.2 Logical Model and Implementation . . . 4

2.3 Database Automation . . . 4

**3 Data Preparation and Export** **5**

3.1 Dataset Generation and Export . . . 5

3.2 Python Preprocessing . . . 5

**4 Data-Mining Methods** **6**

4.1 One Rule (1R) Method . . . 6

4.2 ID3 Decision Tree . . . 6

4.3 Naïve Bayes . . . 7

**5 Model Evaluation** **8**

5.1 Results and Discussion . . . 8

**6 Strategic Model Deployment** **9**

6.1 Deployable Models . . . 9

6.2 Generating Predictions for New Patients . . . 9

6.3 Deployment Perspectives . . . 9

6.3.1 Standalone Python Tool . . . 9

6.3.2 Integration with DBMS or REST Service . . . 9

**7 Mushroom Dataset Analysis (A1)** **10**

7.1 Dataset Analysis . . . 10

7.2 Data Transformation for Orange (.csv to .tab) . . . 10

**8 Application of One Rule (1R) Classifier** **11**

8.1 1R Training and Rule Selection . . . 11

8.2 Generation of the Required Output File . . . 11

**9 Exploratory Analysis using Orange DM** **12**

9.1 Workspace Overview and Data Inspection . . . 12

9.2 Model Evaluation and Prediction Generation . . . 13

# 1. Introduction and Background

## 1.1 Introduction

At MedKnow, ophthalmologists perform eye exams and record clinical data for each patient visit. Every consultation creates a new record that shows the patient's current visual condition and prescription decisions. This ensures medical records are accurate and consistent over time.

Each patient may have multiple consultations, allowing the medical center to track changes in vision and the effectiveness of previous treatments. Each doctor sees several patients in a day, and every consultation includes the registration of the exam observations such as vision prescriptions, tear secretion rate, and the type of contact lenses prescribed.

The patient's age group, is automatically derived from their birth date at the time of consultation. Diagnoses like myopia, hypermetropia, and astigmatism are listed in a DISEASE table, and a single consultation may include multiple diagnoses.

## 1.2 Objectives

The main objectives of this project are to develop a data-driven prototype for operational and strategic support; apply and compare classification algorithms for lens prediction; and transform structured patient data into actionable knowledge for decision-making.

## 1.3 From Data to Knowledge

Data mining follows the DIKW hierarchy, turning data into information and knowledge.

In the MedKnow scenario, raw data consist of patient attributes such as age, tear rate, and lens prescriptions. Through analysis, information emerges as patterns (e.g.patients with reduced tear rate rarely require lenses). Finally, knowledge is established when predictive rules connect these features to lens types, supporting accurate and consistent medical decisions.

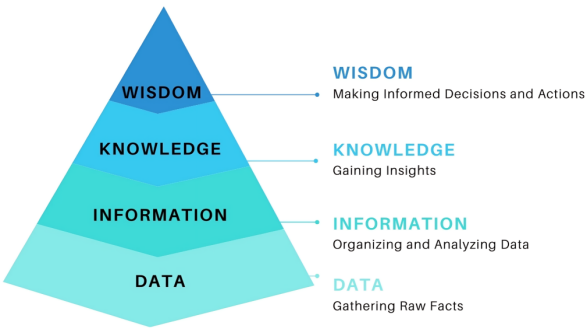



Figure 1.1: The DIKW hierarchy.

Using techniques such as 1R, ID3, and Naïve Bayes, MedKnow can build predictive models for diagnosis and prescription. As new patient data is added, the system can improve its models, creating a feedback loop that transforms daily data into strategic knowledge and captures evolving trends.

# 2. Database Design

The operational model of MedKnow structures clinical data from ophthalmology consultations, ensuring information is consistently stored and retrievable for both operational and analytical purposes.

## 2.1 Conceptual Model (ER Model)

Figure 2.1 shows the Entity–Relationship (ER) diagram capturing MedKnow's workflow. The main entities (`PATIENT`, `DOCTOR`, `DISEASE`, `CONSULTATION`) and their attributes are detailed in the diagram.

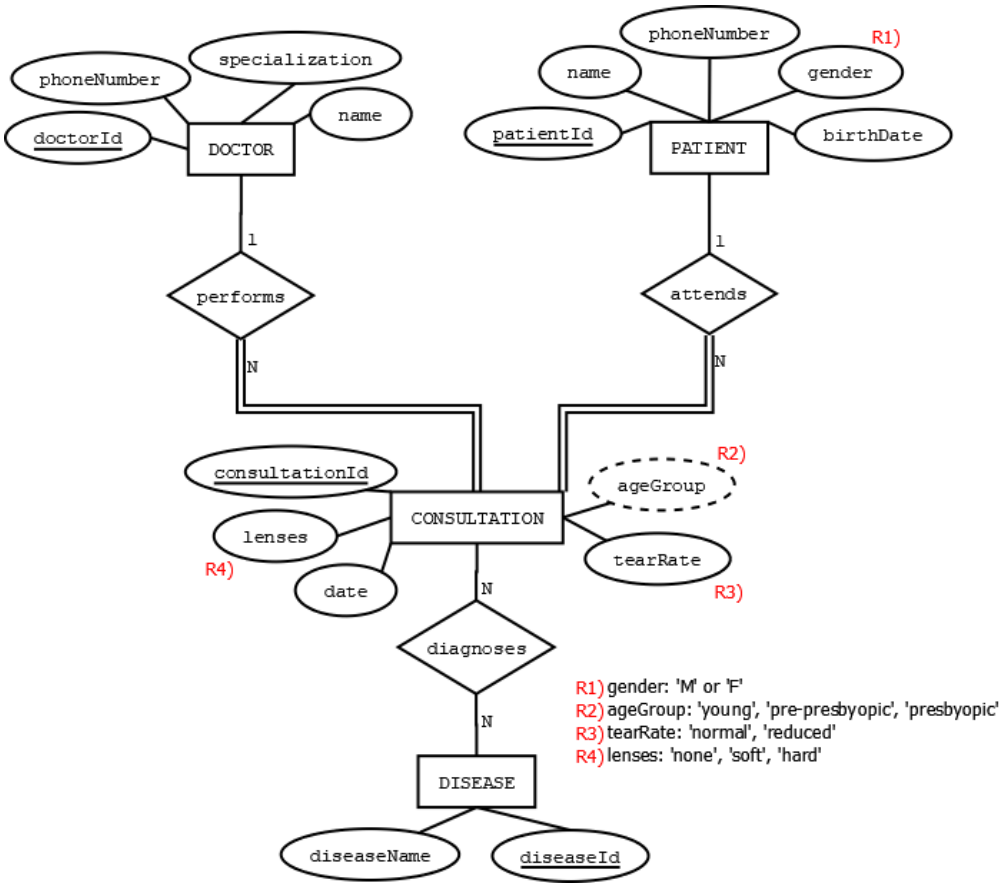



Figure 2.1: Entity–Relationship (ER) diagram.

**Design Decisions:**

- Each doctor may do multiple consultations, and each patient may attend multiple consultations, reflecting real operational workflow.
- Multiple diagnoses per consultation are supported via the associative entity `DIAGNOSIS`, enabling recording of patients with coexisting conditions.

- Attribute constraints (e.g., `gender`, `ageGroup`, `tearRate`, `lenses`) enforce data integrity and consistent categorical encoding.
- Automated derivation of `ageGroup` via a trigger ensures consistent classification of patients based on birth date at the time of consultation.

## 2.2 Logical Model and Implementation

The relational model translates the conceptual schema into PostgreSQL tables with primary and foreign keys enforcing relationships. Referential integrity is maintained via `ON DELETE CASCADE` rules, and the `set_age_group` trigger automatically computes each patient's age group during consultations.

## 2.3 Database Automation

Database creation and population are automated through batch scripts:

- `_go00.bat` — create database
- `_go01.bat` — create schema
- `_go02.bat` — populate tables with sample data
- `_go03.bat` — create views
- `_go04.bat` — export datasets to `scripts/datasets/...`

This automation ensures a consistent and reproducible setup, supporting both regular operations and subsequent data-mining analyses.

# 3. Data Preparation and Export

Building on the operational model and automated database design described in Chapter 2, the MedKnow operational data is consolidated into a clean, analysis-ready dataset suitable for data mining. This process involves extracting relevant attributes from multiple tables and exporting a structured CSV suitable for tools like Orange or Python's `pandas`.

While the initial dataset (`d01_lenses.xls`) provided a small sample of consultations, the MedKnow database has been populated with additional records to provide a richer, more representative dataset. This extended data ensures that the classification models have sufficient variety for training and evaluation.

## 3.1 Dataset Generation and Export

The main dataset is obtained from PostgreSQL via the `dataset` view, producing one record per consultation with attributes: ageGroup, gender, prescription (excluding astigmatism), astigmatic, tearRate, and lenses.

An auxiliary view, `dataset_3row`, formats the CSV for Orange by adding two metadata rows: the second specifies data types (`d` for discrete), and the third marks `lenses` as the target class. Exporting this view allows Orange to import the dataset directly.

Table 3.1 shows a sample of the exported CSV.

| **ageGroup** | **gender** | **prescription** | **astigmatic** | **tearRate** | **lenses** |
|---|---|---|---|---|---|
| young | F | hypermetrope | no | reduced | none |
| young | M | myope | yes | reduced | hard |

Table 3.1: Sample of the exported dataset for data mining.

## 3.2 Python Preprocessing

Before training, the CSV is preprocessed in Python: missing categorical values are filled with *'none'*, features and the target are label-encoded using `LabelEncoder`, and the data is split into training and testing sets (70%/30%). This encoded dataset is used for training and evaluating the algorithms.

# 4. Data-Mining Methods

Three classification algorithms were applied to the MedKnow dataset to predict the type of contact lenses to prescribe based on consultation records: One Rule (1R), ID3 Decision Tree, and Naïve Bayes. All models were trained in Python on numerically encoded categorical data.

## 4.1 One Rule (1R) Method

The One Rule (1R) algorithm is a simple, interpretable classification method that builds a single decision rule based on **one attribute**. It works as follows: for each feature in the training dataset, all possible values are examined; for each value, the most frequent class of the target variable (lens type) is determined; the overall accuracy of this simple rule on the training data is calculated; finally, the feature whose rule achieves the highest training accuracy is selected as the classifier.

In this project, 1R was implemented manually in Python. The model iterates over all features, constructs frequency-based rules for each value, and selects the attribute with the best predictive power. For the MedKnow dataset, the attribute **prescription** provided the highest accuracy, yielding rules such as:

$$\texttt{hypermetrope} \rightarrow \texttt{none}, \quad \texttt{myope} \rightarrow \texttt{soft}, \quad \texttt{none} \rightarrow \texttt{soft}$$

This indicates that a patient's prescription alone is often sufficient to predict the appropriate lens type.

1R's simplicity provides a transparent baseline and makes it easy to understand which features are most influential.

## 4.2 ID3 Decision Tree

ID3 (Iterative Dichotomiser 3) is a recursive decision tree algorithm that splits the dataset based on the feature providing the **highest information gain** (reduction in entropy). The algorithm proceeds until all examples in a branch belong to the same class or no further splitting is possible.

In this project, ID3 was implemented using scikit-learn's `DecisionTreeClassifier` with the `criterion='entropy'` parameter to replicate the information gain splitting of the original ID3.

The resulting tree captures interactions between features such as age group, tear rate, and astigmatism, providing an interpretable structure for clinical decision-making. Figure 4.1 illustrates the tree learned from the MedKnow dataset.

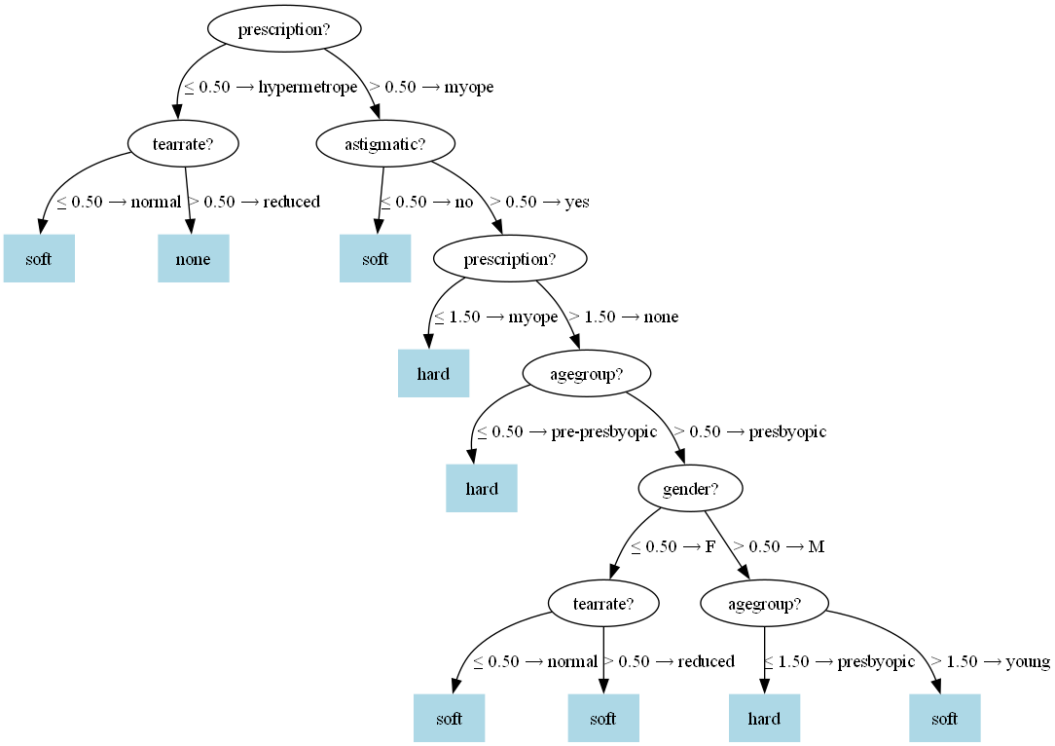



Figure 4.1: ID3 decision tree trained on the MedKnow dataset.

ID3 considers **combinations of attributes**, which allows it to capture complex patterns in the data and improve accuracy when multiple features interact to determine the lens type.

## 4.3 Naïve Bayes

The Naïve Bayes classifier is a probabilistic model based on **Bayes' Theorem**, assuming conditional independence between features given the class:

$$P(C \mid X) = \frac{P(X \mid C) \cdot P(C)}{P(X)}$$

For this dataset, the categorical version (`CategoricalNB`) was used. It computes the chance of each lens type given each feature value, and predicts the class with the highest probability according to Bayes' theorem.

Table 4.1 shows an example of conditional probabilities for the `prescription` feature.

| | hypermetrope | myope | none |
|---|---|---|---|
| hard | 0.0667 | 0.6000 | 0.3333 |
| none | 0.8000 | 0.1000 | 0.1000 |
| soft | 0.2500 | 0.4167 | 0.3333 |

Table 4.1: Conditional probabilities for lens type given `prescription`.

Naïve Bayes is computationally efficient, robust to small datasets, and suitable for real-time applications.

# 5. Model Evaluation

Model performance was evaluated on a 30% test set (18 unseen consultations) using four standard metrics: accuracy, precision, recall, and F1-score, along with execution time. Accuracy measures overall correctness; precision and recall assess per-class prediction quality; and the F1-score provides a balanced summary of both.

## 5.1 Results and Discussion

Table 5.1 summarizes the results. Naïve Bayes was the fastest, ID3 the slowest, though all models were computationally efficient.

| Model | Accuracy | Precision | Recall | F1-score | Time (s) |
|---|---|---|---|---|---|
| 1R | 0.56 | 0.38 | 0.56 | 0.45 | 0.00096 |
| ID3 | 0.72 | 0.72 | 0.72 | 0.72 | 0.00106 |
| Naïve Bayes | 0.72 | 0.72 | 0.72 | 0.72 | 0.00086 |

Table 5.1: Performance metrics for the 1R, ID3, and Naïve Bayes models.

| True | 1R | | | ID3 | | | Naïve Bayes | | |
|---|---|---|---|---|---|---|---|---|---|
| Class | Hard | None | Soft | Hard | None | Soft | Hard | None | Soft |
| Hard | 0 | 0 | 6 | 4 | 0 | 2 | 3 | 0 | 3 |
| None | 0 | 6 | 0 | 0 | 6 | 0 | 0 | 6 | 0 |
| Soft | 0 | 2 | 4 | 3 | 0 | 3 | 2 | 0 | 4 |

Table 5.2: Confusion matrices (columns = predicted classes).

All models correctly identified the `none` lens class, while 1R struggled with `hard` and `soft` lenses. ID3 and Naïve Bayes achieved identical accuracy (0.72), clearly outperforming 1R.

The 1R model, based on a single feature, is simple and interpretable but limited in capturing feature interactions such as tear rate or age group.

Among the higher-performing models, ID3 offers an interpretable decision tree that supports explainable clinical decisions and model transparency. Naïve Bayes, on the other hand, is faster and produces probabilistic outputs suitable for real-time or uncertain scenarios.

In conclusion, ID3 is preferable when explainability is required, while Naïve Bayes is ideal when efficiency and probabilistic recommendations are priorities. The 1R algorithm remains a lightweight baseline for quick or interpretable rule generation.

# 6. Strategic Model Deployment

The *SoftKnow* prototype delivers deployable predictive models that extend MedKnow's operational database with intelligent decision support. The trained models can automatically recommend the appropriate contact lens type based on patient consultation features

## 6.1 Deployable Models

Two deployable solutions were developed:

- The **1R model**, offering a simple and fully interpretable rule-based approach.
- The **ID3 and Naïve Bayes models**, capable of capturing more complex relationships among patient features.

All trained models, together with their encoders, are stored in the `/artifacts` directory as serialized files (`.pkl` for Python objects). The use of Python's `pickle` module ensures that models and preprocessing objects can be reloaded exactly as they were at training time, enabling consistent future predictions without the need for retraining.

## 6.2 Generating Predictions for New Patients

New patient data is supplied in CSV format and processed using the same feature encoders employed during training. Missing values are handled, and unseen categories are mapped to 'none'.

The predictions are made by loading the previous trained models. All predicted labels are decoded to the original lens type categories and returned alongside the input features, providing a complete and interpretable result.

## 6.3 Deployment Perspectives

### 6.3.1 Standalone Python Tool

The solutions run locally as a Python command-line tool. MedKnow staff can supply a CSV containing patient features and instantly receive the predicted lens types. This deployment requires minimal infrastructure and is suitable for routine clinical use.

### 6.3.2 Integration with DBMS or REST Service

For seamless integration into MedKnow's information system, the solutions could be used in the database or exposed through a REST API. In this case, new patient records inserted into the DBMS could automatically trigger the prediction process, with results written back to the database. This approach enables real-time support during consultations.

# 7. Mushroom Dataset Analysis (A1)

This chapter presents the application of the project's classification methods to the UCI *Mushroom* dataset. The goal is to predict whether a mushroom is edible or poisonous based on 22 categorical features. This dataset provides a suitable test case for implementing the 1R algorithm on a larger dataset and for experimenting with additional classifiers in the Orange Data Mining (DM) tool.

## 7.1 Dataset Analysis

The dataset from the UCI Machine Learning Repository contains 8,416 instances (after removing a non-data footer row). Each instance represents one mushroom and is labelled with the target attribute:

- **Target Variable:** `class`, with two possible values: edible (`e`) and poisonous (`p`). The distribution is relatively balanced (53.3% edible, 46.7% poisonous).
- **Features:** The dataset includes 22 nominal attributes describing physical characteristics such as `cap-shape`, `odor`, `gill-color`, and `habitat`.
- **Missing Values:** A challenge in this dataset is the presence of missing values. The stalk-root attribute (feature 11) contains 2,480 missing values, denoted by a ? character.

## 7.2 Data Transformation for Orange (.csv to .tab)

To enable analysis in Orange DM, the original `dataset_long_name_ORIGINAL.csv` was converted into Orange's `.tab` format using a Python script. The transformation followed Orange's required three-line header structure:

- **Line 1 - Attribute Names:** A tab-delimited list of all column names (`class`, `cap-shape`, `cap-surface`, ...).
- **Line 2 - Attribute Types:** Since all attributes in this dataset are nominal, this line consisted of `discrete` repeated 23 times.
- **Line 3 - Target Specification:** A line indicating the target variable. The entry for the `class` column was set to class, while all other feature columns were left blank.

The script loaded the CSV while preserving the `?` symbol as a categorical value, removed the footer row, generated the required header, and wrote the data using tab delimiters. The resulting file, `dataset_long_name_PTS_INPUT_v01.tab`, complies fully with Orange DM's format, correctly identifying the target variable and all feature types.

# 8. Application of One Rule (1R) Classifier

This chapter describes how the previously implemented 1R classifier was applied to the Mushroom dataset and how the required evaluation file was produced.

## 8.1 1R Training and Rule Selection

Using the preprocessing and encoding procedures described earlier, the Mushroom dataset was fully transformed into numerical form: all 22 categorical features and the target attribute were label-encoded. The classifier was trained on **all available data**, without a train–test split.

During training, the algorithm evaluated each feature independently. For every feature, it constructed a simple rule that predicted the most frequent class associated with each of its values. The total classification error of each candidate rule was then computed, and the feature with the lowest error was selected as the final "one rule" used by the model.

## 8.2 Generation of the Required Output File

A dedicated procedure was implemented to analyse the behaviour of the selected rule over the full dataset and to generate the output file in the specified format.

Instead of only reporting the final rule, the procedure iterated through every encoded row of the dataset and, for each instance:

1. identified the true class and the class predicted by the chosen 1R rule;
2. grouped the results by the pair *(selected attribute value, true class)*;
3. counted, for each group:
    - the total number of samples matching that combination;
    - the number of those samples misclassified by the rule.

Once these statistics were computed, all encoded values were converted back into their original categorical names using the stored label encoders. The results were then written to the output file `oneR_OUTPUT.txt` in the required format:

$$(\texttt{attribute}, \texttt{valueAttr}, \texttt{valueTarget}) : (\texttt{error}, \texttt{total})$$

This file provides a detailed breakdown of how the selected 1R rule performs for every combination of feature value and true class, offering a transparent view of the model's predictive behaviour and error distribution.

# 9. Exploratory Analysis using Orange DM

Building on the data preparation in Section 7.2, this chapter presents the data mining workflow developed in Orange Data Mining to analyze the mushroom dataset and compare two classifiers: Decision Tree and Random Forest.

## 9.1 Workspace Overview and Data Inspection

An Orange workspace was created to visually explore the dataset and perform classifier evaluation. The workflow (Figure 9.1) begins with the **File** widget to load the `.tab` file.

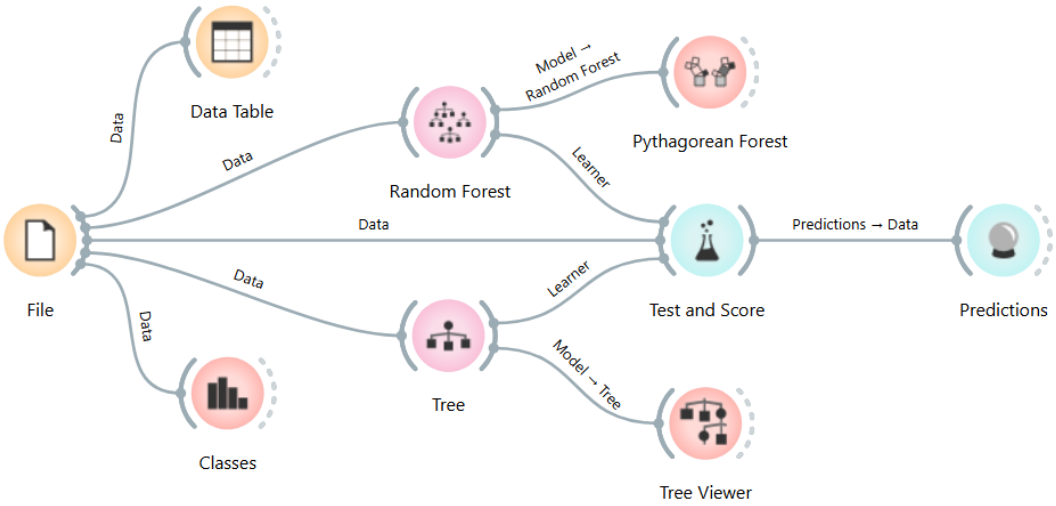



Figure 9.1: Orange DM workflow for mushroom data analysis.

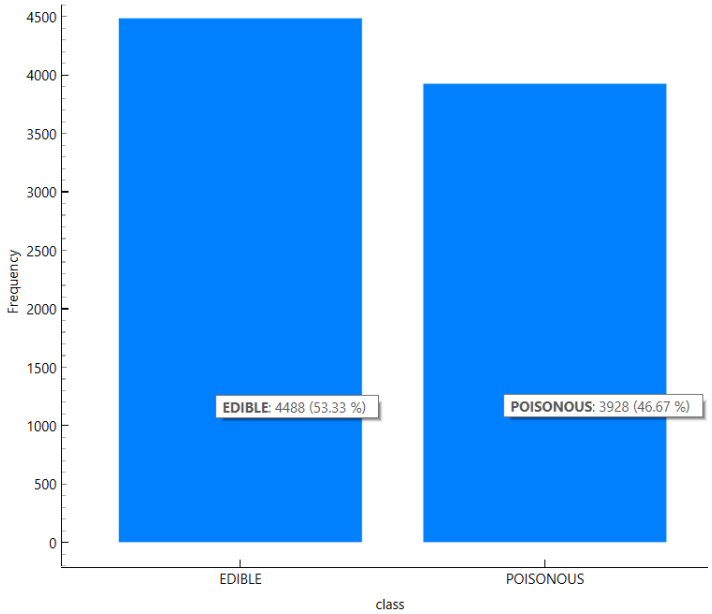



Figure 9.2: Class distribution of edible vs. poisonous mushrooms.

- Total Instances (Item 4a): The **Data Table** widget confirmed that the dataset contains all 8,416 instances.

- Class Distribution (Item 4b): Using the **Distributions** widget, the target variable `class` was inspected, confirming 4,488 edible (e) and 3,928 poisonous (p) mushrooms (Figure 9.2).

## 9.2 Model Evaluation and Prediction Generation

To evaluate model performance, the dataset and classifiers were connected to the **Test and Score** widget using 5-fold cross-validation. The resulting metrics are shown in Table 9.1.

| Model | AUC | CA | F1 | Precision | Recall | LogLoss |
|---|---|---|---|---|---|---|
| Tree | 0.983 | 0.986 | 0.986 | 0.986 | 0.986 | 0.972 |
| Random Forest | **1.000** | **1.000** | **1.000** | **1.000** | **1.000** | **1.000** |

Table 9.1: Performance metrics for Tree and Random Forest.

### Decision Tree (Item 4c)

A **Tree** widget was connected to the evaluation flow and inspected via the **Tree Viewer** (Figure 9.3). The tree captures feature interactions, in particular the strong predictive role of `odor`. Although highly accurate (CA = 0.986), it does not achieve perfect separation.

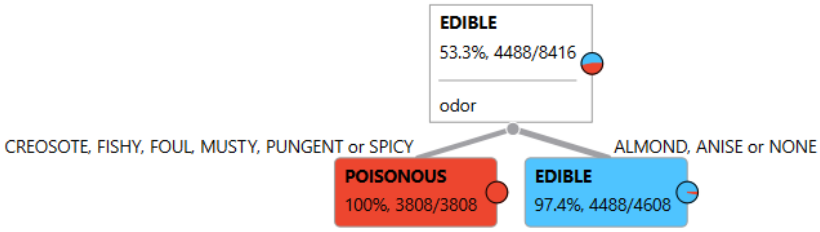



Figure 9.3: Decision tree visualization for mushroom classification.

### Random Forest (Item 4d)

A Random Forest is a group of decision trees working together. Each tree is trained on a random subset of the data and features, so every tree sees a different view of the problem. When predicting a new example, each tree "votes" for a class, and the class with the most votes becomes the final prediction. This approach reduces mistakes caused by individual trees and usually achieves higher accuracy than relying on a single tree.

In Orange, a **Random Forest** widget was added in parallel to the tree model. As shown in Table 9.1, it achieved perfect accuracy (CA = 1.000) making it the most suitable choice for the "FungiData" predictive needs. The **Pythagorean Forest** widget was used to visualize structure and feature importance across the ensemble.

### Prediction Generation (Item 4e)

Finally, the outputs of both classifiers were routed to the **Predictions** widget, which displays the true vs. predicted classes for each test fold. The Random Forest produced 100% correct predictions, while the Decision Tree showed a few misclassifications.